

1,024-query SQL follow-up Storage benchmark analysis

Manoj Khatri / HPCAI-lab | Perlmutter | 2 October 2026

Internal progress report. This document summarizes saved experiments and exploratory analysis. It is not a publication manuscript. It supplements the original eight-run pilot report.

New milestone verified

New measurements	Experimental coverage	Validation
180 measured + 36 warmups	3 SQL deployments x 2 workloads x 2 tiers x 3 client counts; 5 measured repetitions	216 full JSON trials; 437,184 latency samples; CSV, plans and source bytes reconciled

Every trial wrote 1,000,000 records in 1,000-record batches and ran 1,024 queries returning 100 rows each. Native SQLite, Shifter SQLite and PostgreSQL ran sequentially on one node per workload. The earlier InfluxDB pilots provide the 1,024-query reference; no new InfluxDB jobs were run.

What the follow-up establishes

- 1. Storage benefit still depends on concurrency.** Native SQLite metadata write throughput has a tmpfs/Lustre mean ratio of 1.84 at one client and 0.90 at 64. The 64-client geometric ratio is 0.90, with a pointwise interval [0.75, 1.08]; parity remains inside that interval.
- 2. Equal query counts produce very short SQL read phases.** Across the 180 new measured trials, 120 read phases lasted less than 50 ms; all 60 trials with 64 clients fall in that group. Each 64-client worker issues only 16 queries. These observations describe short bursts.
- 3. The reported read rate depends on experiment context.** PostgreSQL metadata reads on Lustre at 64 clients average 65.6k queries/s here versus 200.1k in the 64,000-query pilot. Allocation and some source files also changed, so this 3.05x difference cannot be attributed solely to query count.

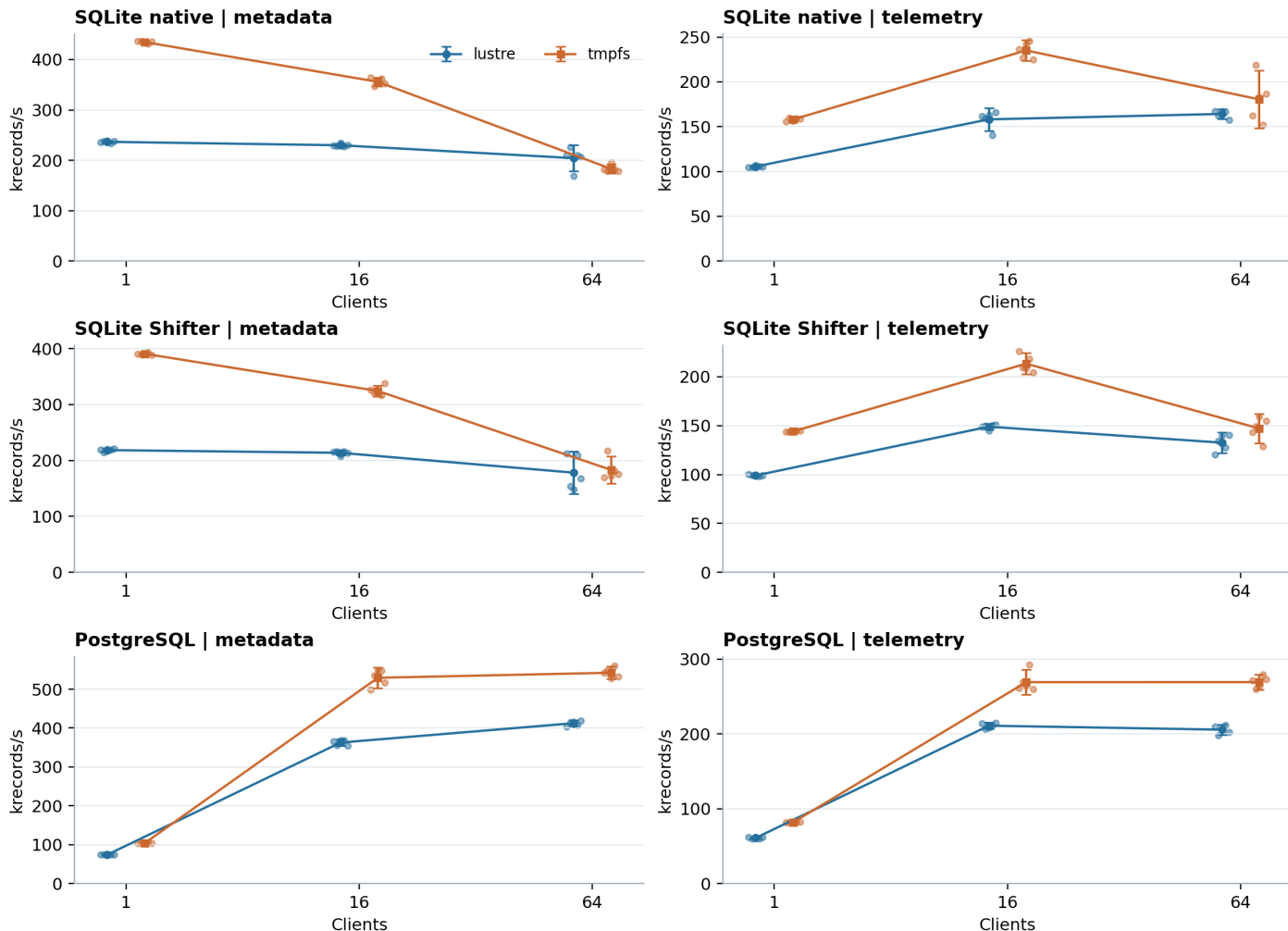
Metadata writes: tmpfs / Lustre mean throughput

Deployment	1 client	16 clients	64 clients
SQLite native	1.84x	1.55x	0.90x
SQLite Shifter	1.79x	1.52x	1.03x
PostgreSQL	1.40x	1.46x	1.31x

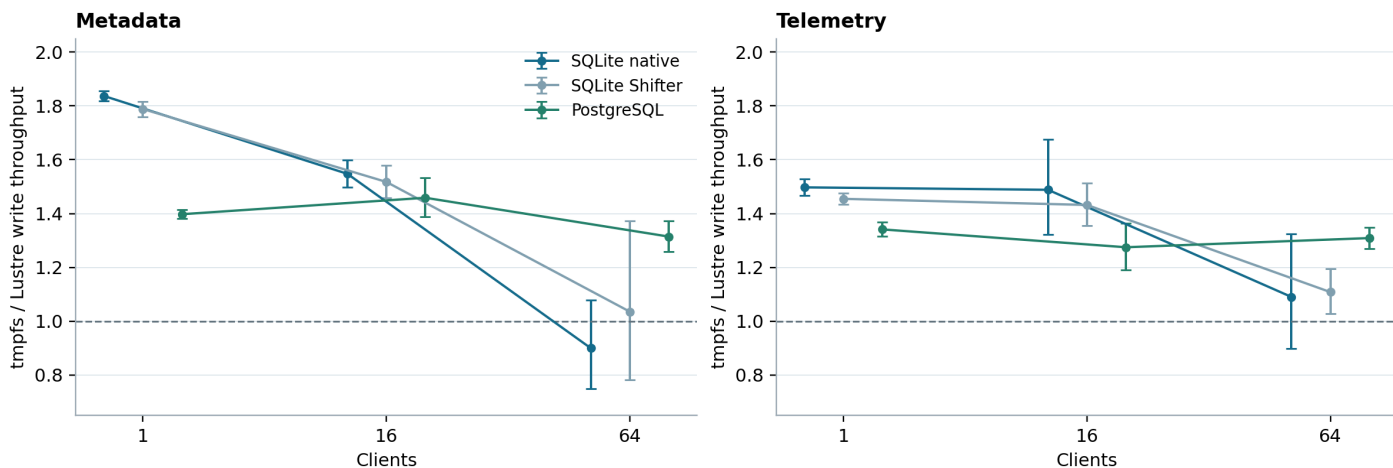
Adding these runs to the original baseline gives 420 distinct measured trials and 84 warmups. Separate profiling/overhead studies are outside these counts. The experiment sets are not pooled as independent replicates.

Write behavior and storage gains

New SQL results only. Points are the five measured trials; lines are arithmetic means. Whiskers are pointwise 95% t intervals conditional on each allocation. Panel scales differ.



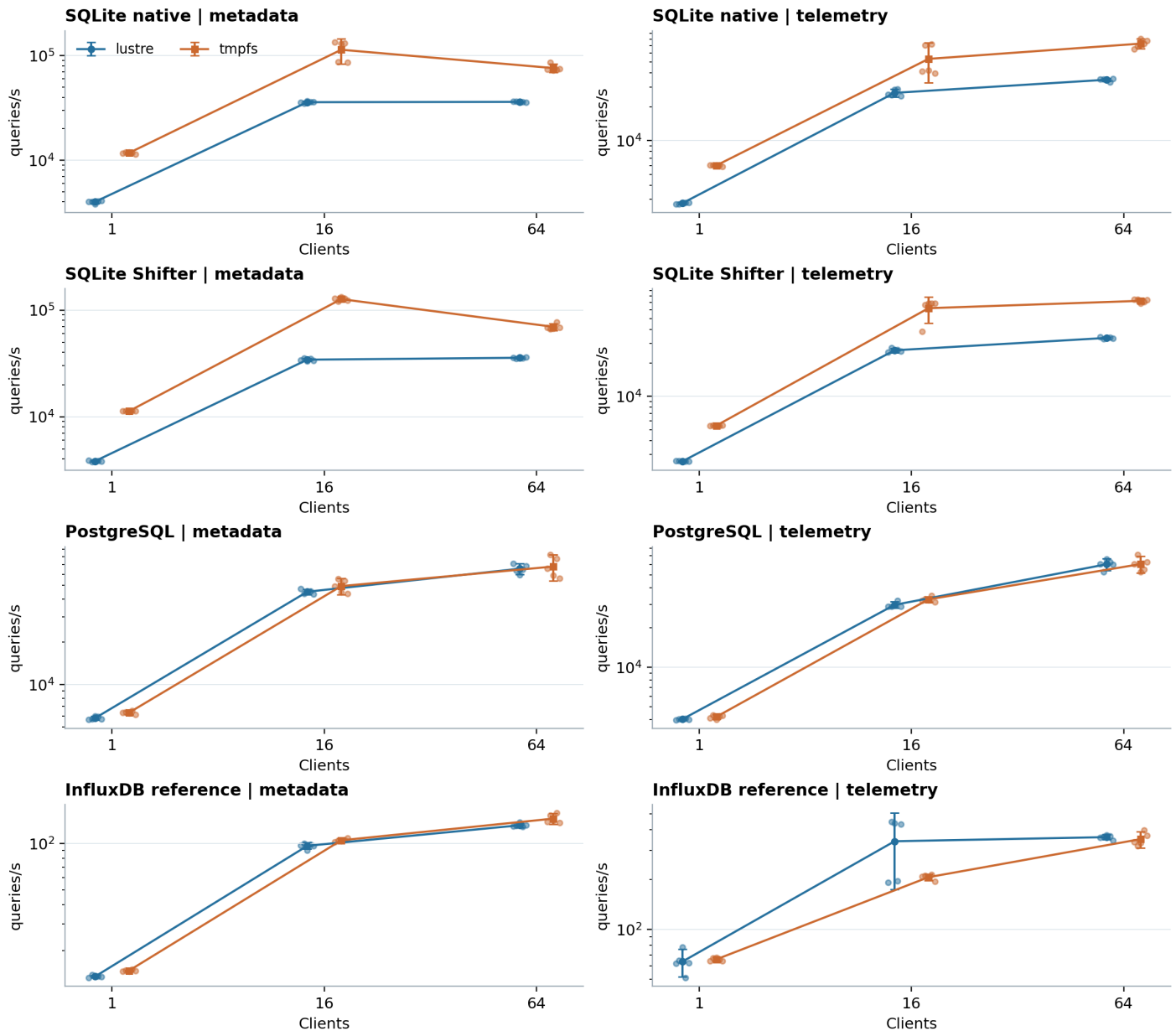
All plotted SQL write rates include the explicit final checkpoint. SQLite uses WAL/NORMAL; PostgreSQL records fsync, full_page_writes and synchronous_commit enabled. These are deployment-specific boundaries and do not establish matched durability.



Lower plot: geometric means of five repetition-matched tmpfs/Lustre ratios, with log-t intervals. Dashed line denotes parity. The intervals belong to geometric estimates, while page 1 reports ratios of arithmetic means.

Warm reads with equal query counts

All panels use 1,024 queries per trial, with the same logical query-selection scheme and 100 returned rows per query. SQL panels are new; InfluxDB panels reuse the earlier pilot on separate allocations. Logarithmic y axes and panel-specific scales.



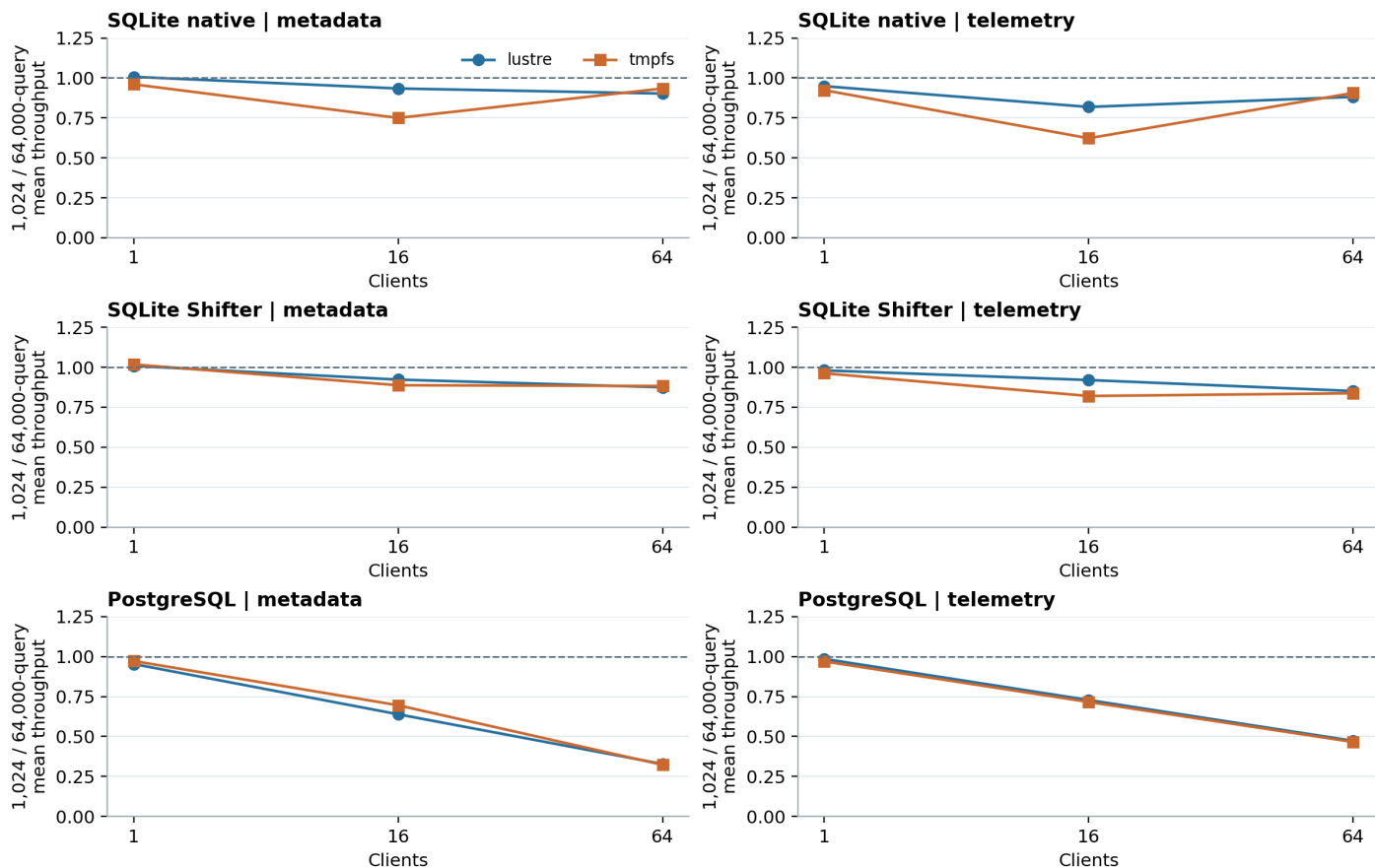
Points are individual trials; whiskers are pointwise 95% t intervals around mean queries/s. Query protocols and physical mappings differ: SQLite is embedded, PostgreSQL uses a local Unix socket, and InfluxDB uses HTTP/Flux with pivot/group/sort operations in this adapter. These figures characterize those implemented paths.

The read timer starts when the parent releases the ready workers and ends at the last worker completion. It excludes worker connection/setup, but includes release/wakeup, query selection, calls, and returned-key validation. Individual request latency times only the backend method. These two timing scopes must remain separate.

Equal counts remove the original count mismatch. They do not equalize phase duration, engine work, versions, server state, or durability; no isolated container-overhead or general database ranking follows from these panels.

Read length and allocation context

Descriptive ratios of arithmetic mean throughput: new 1,024-query runs divided by earlier 64,000-query runs. Both sets have five measured repetitions per condition. No causal confidence interval is assigned to this cross-allocation contrast.



PostgreSQL at 64 clients

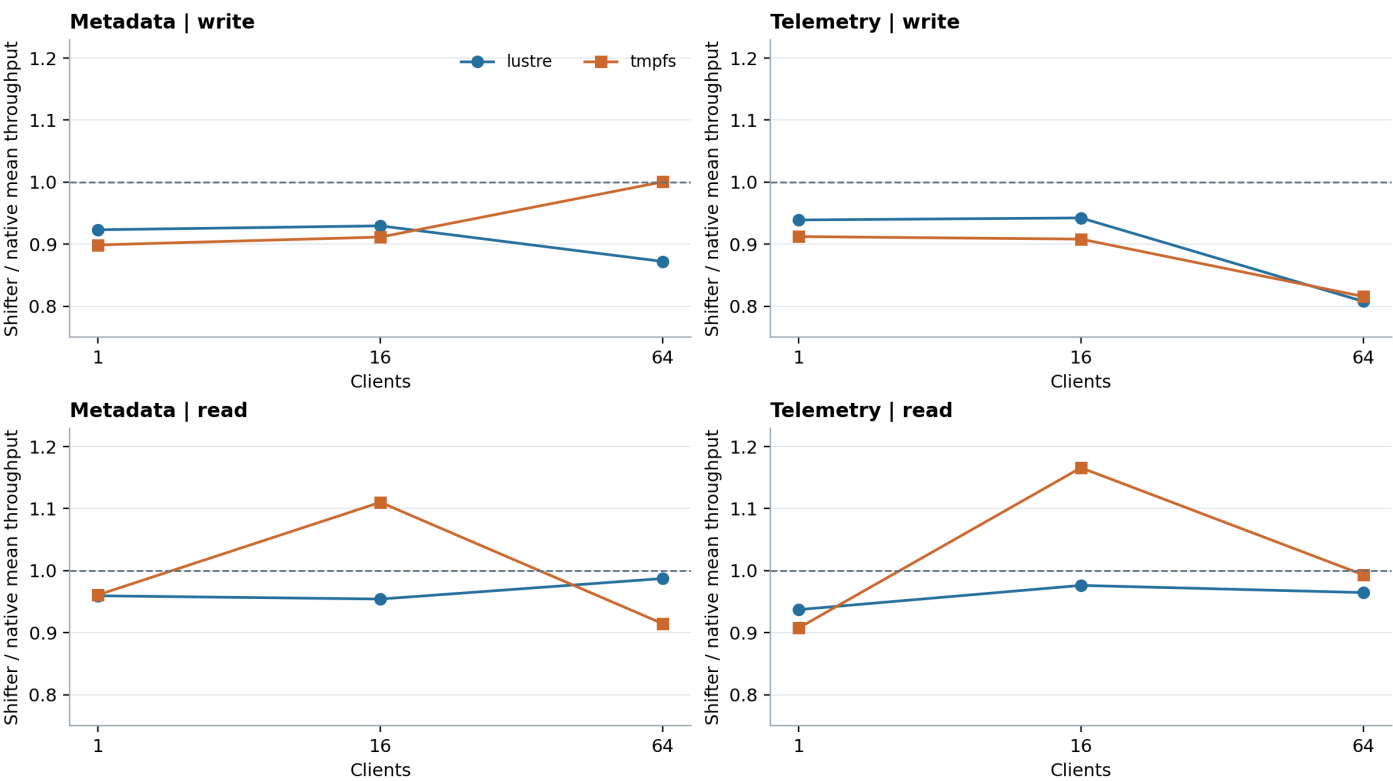
Workload / tier	1,024 queries: kQ/s	64,000 queries: kQ/s	1,024 read phase (ms)
metadata / lustre	65.63	200.12	15.66
metadata / tmpfs	68.04	210.80	15.39
telemetry / lustre	60.42	127.80	17.04
telemetry / tmpfs	60.59	130.13	17.08

The metadata rates in the longer pilot are about 3.05-3.10x those in the new short runs; telemetry rates are about 2.12-2.15x. This is consistent with sensitivity to duration and synchronization, but it does not identify their individual contributions.

Confounding remains: the YAML settings differ only in read_queries, but allocations changed. SQLite also has different saved hashes for its controller, write coordinator and read worker; PostgreSQL differs in the factory, coordinator, lifecycle and controller. All changed filenames are listed in validation.json. Requests are a fixed deterministic prefix, not an equal-duration workload.

SQLite runtime comparison on the same node

New runs: Shifter/native ratios of arithmetic mean throughput. Both deployments used the same source snapshot and node within each workload. They ran as separate sequential blocks in fixed order, so time/order effects remain. Ratios are descriptive.



Deployment	Python	SQLite	Execution context
Native	3.11.7	3.44.2	First deployment block
Shifter	3.11.15	3.46.1	Second deployment block

Shifter mean write throughput is lower in 11 of 12 conditions. The largest observed gap is about 19.2% for telemetry on Lustre at 64 clients. Metadata writes on tmpfs at 64 clients are effectively equal in their means (ratio 1.0004), with substantial trial variability.

For reads, Shifter has higher means at 16 clients on tmpfs: about 11.0% for metadata and 16.6% for telemetry. These short phases are variable; native throughput CVs in those two conditions are 21.8% and 31.1%. This does not establish a container advantage.

This design controls the compute-node identity for the new SQL comparison. Isolating container overhead still requires matched interpreter/database builds and a counterbalanced or randomized runtime order within allocation.

Methods, provenance and remaining work

Workload	Slurm job / node	Order within allocation
Metadata	59220317 / nid005765	SQLite native -> SQLite Shifter -> PostgreSQL
Telemetry	59220318 / nid006712	SQLite native -> SQLite Shifter -> PostgreSQL

Source snapshot: 63c227bc25fc394d3a3752f3fe4f19ebc78fff65. All six source manifests agree, and every included source file matches its SHA-256 hash. The recorded commit is retained; a full Git object database is not part of the export. Native clients use Python 3.11.7; PostgreSQL is 16.12 in a pinned Shifter image.

Validation and evidence scope

The new archive passes ZIP CRC and all 280 recorded file checksums. All 216 trial JSONs agree with CSV/config/plan identities, counts, finite timings, worker distributions, CPU sums and rate identities. Mean and p50/p95/p99 latencies were recomputed from 437,184 batch/query samples. Both logs show all 108 trials and the final completion marker. Original databases were not reopened.

The earlier export contains 288 CSV rows plus selected metadata, plans and hashes, without full raw latency samples or historical source bytes. Its internal consistency was rechecked, but its evidence depth differs from the new export. The earlier PostgreSQL/InfluxDB server-profiling experiments are outside this package.

Statistics

Warmups are retained in the archive and excluded from estimates. Each condition has $n=5$ measured trials. Mean intervals use $\text{mean} \pm t(0.975,4) \times \text{sample SD} / \sqrt{5}$. Storage/scaling/interaction contrasts use five repetition-matched log ratios and exponentiated t intervals. All intervals are pointwise and model-based, conditional on the observed allocation; serial dependence or drift may invalidate the nominal coverage.

No outliers were removed and no multiplicity-adjusted or omnibus significance claim is made. Latency summaries describe trial percentiles, not independent request replicates. Arithmetic mean throughput is the mean of per-trial rates. The old/new read ratios and native/Shifter ratios are descriptive and have no causal-effect interval.

Completed and next

Completed: the 1,024-query SQL comparison, raw-data validation, source verification, uncertainty tables, figures and reproducible offline analysis. The original baseline remains a separate experiment set.

Remaining: mixed workloads; systematic 100,000 and 10 million record coverage; matched runtime builds and documented completion/durability policies; independent allocation repeats with execution-order control; hardware counters and aligned system-level measurements; final research synthesis and artifact preparation.

A focused same-allocation, randomized 1,024-versus-64,000-query comparison would resolve the duration question more directly if sustained read-throughput claims are needed. It should reuse one source/build snapshot and preserve the original query semantics.

Package inputs: read1024-results-20261002T200306Z-67f254d2.zip and ipdps-pilot-analysis.zip. Both originals are included unchanged. Tables expose full precision, provenance, trial variability and the exact source differences.